Week 8 — Errors & Debugging

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Syntax vs. runtime errors
- try / except — catching specific exceptions
- Defensive programming and guard clauses
- pip and virtual environments

Apply — ~30 min

- Code-along: input validation loop
- Code-along: safe file reader
- Code-along: defensive CSV reader starter

Two Kinds of Errors

Syntax error — Python can't even parse the code. Caught before running.

Runtime error — Code parses fine, but crashes when something unexpected happens during execution.

```
SyntaxError: invalid syntax
ValueError: invalid literal for int() with base 10: 'hello'
FileNotFoundError: [Errno 2] No such file or directory
```

try / except

Wrap risky code in `try`. Handle specific errors in `except`.

```
try:
    value = int(input("Enter a number: "))
    print(f"You entered: {value}")
except ValueError:
    print("That's not a valid number.")
```

Common Runtime Exceptions

Exception	When it occurs
`ValueError`	Wrong type/value, e.g. `int("hello")`
`TypeError`	Wrong type for an operation
`KeyError`	Dictionary key doesn't exist
`IndexError`	List index out of range
`FileNotFoundError`	File doesn't exist
`ZeroDivisionError`	Division by zero

Check for Understanding #1

Which exception should you catch when a user enters text instead of a number?

```
age = int(input("Enter your age: "))
```

A) `TypeError`
B) `ValueError`
C) `KeyError`
D) `IndexError`

Defensive Programming

Don't wait for errors — validate inputs before you use them.

```
def calculate_average(numbers):
    if not numbers:
        return 0
    return sum(numbers) / len(numbers)
```

Failing Gracefully

Wrap file reads and external calls in try/except. Return safe defaults on failure.

```
def read_data(path):
    try:
        with open(path, "r") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Warning: {path} not found.")
        return ""
```

Check for Understanding #2

What is the difference between catching exceptions row-by-row vs. wrapping the whole loop in one try/except?

A) No difference — both catch the same errors
B) Row-by-row: one bad row skips cleanly; one try/except around the loop: first error stops all processing
C) Row-by-row is slower
D) You can only use try/except outside loops

Virtual Environments and pip

A **virtual environment** isolates your project's packages so they don't conflict with other projects.

```
python -m venv .venv           # create
source .venv/bin/activate      # activate (Mac/Linux)
.venv\Scripts\activate         # activate (Windows)

pip install requests           # install a package
pip freeze > requirements.txt # save the list
```

Check for Understanding #3

Why do we run `pip freeze > requirements.txt`?

A) To delete unused packages

B) To save the exact versions of all installed packages so others can reproduce the environment

C) To activate the virtual environment

D) To check for package updates

Time to Apply

Let's write code that handles the unexpected.

Mentor 2 takes over

Assignment 8 Overview

Part 1 — Four warmup scripts

- `warmup1.py` — input loop with `ValueError` catch
- `warmup2.py` — safe division with `ZeroDivisionError`
- `warmup3.py` — handle a `FileNotFoundError` gracefully
- `warmup4.py` — set up a venv, install `requests`, freeze to `requirements.txt`

Part 2 — Defensive CSV Reader

- Read a messy CSV row-by-row with try/except per row
- Catch `ValueError` (bad amount) and `KeyError` (missing column)
- Print a summary: rows attempted, parsed, skipped — with details on each skip

Code-Along 1: Input Validation Loop

Keep asking until the user enters a valid number.

```
while True:
    try:
        value = float(input("Enter a number: "))
        print(f"You entered: {value}")
        break
    except ValueError:
        print("That's not a valid number. Try again.")
```

Code-Along 2: Safe File Reader

Return a useful value even when the file doesn't exist.

```
def read_file(path):
    try:
        with open(path, "r") as f:
            return f.readlines()
    except FileNotFoundError:
        print(f"File not found: {path}")
        return []

lines = read_file("../data/missing.txt")
print(f"Lines read: {len(lines)}")
```

Code-Along 3: Defensive CSV Row Parser

Process each row inside its own try/except.

```
import csv

skipped = []
clean = []

with open("../data/messy_data.csv", "r") as f:
    for i, row in enumerate(csv.DictReader(f), start=2):
        try:
            amount = float(row["amount"])
            clean.append({"name": row["name"], "amount": amount})
        except (ValueError, KeyError) as e:
            skipped.append(f"Row {i}: {type(e).__name__} — {e}")
```

Extension Challenge

Ungraded extension: Add a check for rows with extra columns.

`csv.DictReader` stores extra columns under the key `None`. Before the try/except block, add:

```
if None in row:
    skipped.append(f"Row {i}: extra column detected")
    continue
```

This is the technique described in the assignment hint.

Wrap-Up

Today you learned:

- Syntax vs. runtime errors — and when each occurs
- `try` / `except` for catching specific exceptions
- Defensive programming: guard clauses and graceful failure
- Virtual environments and `pip freeze`

This week: Complete warmups 1–4 and the Defensive CSV Reader. Submit via pull request.

Next week: External APIs — your programs will reach out to the internet for live data.